Development of Online Alumni Data Management System

A Mini-Project submitted for the fulfilment of Advanced

Database Management System Laboratory (CS15204)

by

SUSHIL KUMAR GUPTA
Roll Number: **B230034CS**
3-Year B. Tech. (C.S.E.)

Under the Supervision of
Dr. Sangram Ray
Associate Professor

Department of Computer Science and Engineering
National Institute of Technology Sikkim
Ravangla 737139, Sikkim, India

DECEMBER 2025

CERTIFICATE

This is to certify that the project report entitled "**DEVELOPMENT OF ONLINE ALUMNI DATA MANAGEMENT SYSTEM**" submitted by **Sushil Kumar Gupta** bearing Roll No. **B230034CS** is an authentic report for the work carried out under my supervision as a part of Advanced Database Management System Laboratory during the Odd Semester, 2025-26 at National Institute of Technology Sikkim, Ravangla-737139, Sikkim.

Dr. Sangram Ray
Associate Professor
Department of Computer Science and Engineering
National Institute of Technology, Sikkim
Ravangla- 737139, Sikkim, India

ACKNOWLEDGEMENT

First and foremost, I would like to thank the Almighty God for giving me the strength, knowledge, ability and opportunity to undertake this project study and to persevere and complete it satisfactorily.

I extend my deepest gratitude to my supervisor, **Dr. Sangram Ray**. It has been an honor for me to do a project under his supervision and sincerely thank him for his excellent guidance, support and motivation in every step throughout the whole period of my project.

I specially thank everyone, for their valuable support, patience, critical comments and detailed review during the period of my project. They always inspired me, willing to help and give their best suggestions.

Finally, I am indeed grateful to all the faculty members, Ph.D. scholars, and staff of the National Institute of Technology Sikkim for providing an environment conducive to my project.

Sushil Kumar Gupta
B230034CS
5th Semester, B. Tech. (CSE)
Department of Computer Science and Engineering

INDEX

Serial No.	Chapters	Page No.
1	**Abstract**	5
2	**Introduction**	6
3	**Problem Description**	7
4	**System Specifications**	8
5	**Problem Formulation**	9-33
	5.1 Existing System	9
	5.2 Objective	10
	5.3 Proposed System	11
	5.4 Entity-Relationship Diagram	12
	5.5 Database Schema	13-22
	5.6 Implementation	23
	5.7 Description of Modules	24
	5.8 Execution Snapshots	25-33
6	**Applications**	34
7	**Conclusion and Future Scope**	35
8	**References**	36

CHAPTER - 01

ABSTRACT

This project is designed to store and manage information about an institution's graduates (alumni) in a centralized, web-based database. Administrators can add new alumni and update their personal details, degrees, achievements, projects, job history, and donation records through simple web pages. The frontend is built with React.js using Vite for fast development. The backend runs on Node.js which connects to a MySQL database. Together, these technologies allow for a responsive interface and efficient data processing. The goal is to make it easy for the college to track and update alumni information and contributions from anywhere, reducing paper records and manual errors.

CHAPTER - 02

INTRODUCTION

Alumni management refers to the systematic approach of building and maintaining strong relationships between an institution and its former students. An Alumni Data Management System automates this process by organizing alumni details (such as names, contact info, education background, careers, and donations) in tables, instead of paper forms or scattered spreadsheets. By having a centralized database, the institution can easily communicate with alumni, track their achievements, and encourage support (for example, through donations or mentorship).

In this project, the user interface is built with React.js. React is an open-source JavaScript library that efficiently renders web pages from reusable components. The backend uses Node.js with the Express framework. Node.js is a server-side JavaScript runtime that lets us run code on the server. Express adds tools for building web APIs. Finally, MySQL is used for the database; it's an open-source relational database system that stores data in tables linked by keys . This combination (a modern web frontend, a JavaScript backend, and MySQL) allows quick development and reliable storage of alumni data, making the system easy to use and maintain

CHAPTER - 03

PROBLEM DESCRIPTION

Many colleges today still keep alumni information in separate files or Excel sheets. This can cause problems: records get lost, duplicated, or hard to update. For example, if an alumnus changes their address or job, the old data might stay on paper and not get updated elsewhere. It's also hard to search or sort information when it is kept manually.

Our project solves these issues by providing an online database system where all alumni data is stored in one place. Instead of paper, administrators and alumni use a web interface to enter or view records. This ensures that whenever someone adds or edits a record (like a new degree or donation), the database updates immediately.

Overall, the system aims to replace manual data handling with an automated approach so that data entry is easier and searching for alumni information is fast and error-free.

CHAPTER - 04

SYSTEM SPECIFICATIONS

The system uses modern web technologies on both client and server sides:

- **React.js (Frontend)** - An open-source JavaScript library for creating user interfaces. It builds pages from components, making development faster. We use Vite as the build tool, which is optimized for speed and developer experience. Vite enables quick page reloads during development.

- **HTML/CSS** - Standard web markup and styles are used within React components to layout and style the pages for alumni, using simple, clean design for easy use.

- **Node.js (Backend)** - A JavaScript runtime that runs on the server. It can handle many requests efficiently by using an event-driven, non-blocking model.

- **Express.js Framework** - A minimal and flexible web framework for Node.js that provides features for creating APIs and web pages. We use it to define URLs (routes) for our application (for example, /addAlumni, /getDonations, etc.).

- **MySQL Database** - An open-source relational database management system to store our data tables. All records (Alumni, Degrees, Jobs, etc.) are stored in MySQL tables.

- **Web Browser** - The system is accessed via any modern web browser (such as Chrome, Firefox, Edge). JavaScript must be enabled on the client side for interactive pages.

- **Development Tools** - Code is written in JavaScript and JSX (for React). We use VS Code as the editor, and npm (Node Package Manager) to install required packages (like React, Express, and MySQL drivers).

These technologies work together: React/Vite handle the user interface on the client, and Node/Express handles requests on the server, communicating with MySQL to save or retrieve data. For example, when a user submits a form to add a new alumnus, React collects the input and sends it to the Node backend (which then uses MySQL queries to insert it into the database).

CHAPTER - 05

PROBLEM FORMULATION

5.1 Existing System

In many institutions today, alumni information is still managed using manual or semi-manual methods. Most colleges store alumni details in paper files, Excel sheets, or separate departmental records. This makes it difficult to maintain accurate, updated, and complete information about former students. When alumni change their contact details, jobs, or educational qualifications, the updates often do not reach the institution, resulting in outdated records.

Existing systems usually lack a centralized database for alumni. Different offices maintain separate copies of the same data, causing mismatches. Because of this, searching for a specific alumnus, verifying achievements, or viewing donation history becomes time-consuming and inefficient.

After analyzing the existing manual and semi-digital systems, we identified the following problems and weaknesses:

- **Scattered and Unorganized Data:** Alumni information is usually stored in different files, spreadsheets, or registers. This makes data retrieval slow and difficult. When the number of alumni grows, it becomes even harder for staff to keep track of everything properly.
- **No Centralized Repository:** There is no single platform that stores all alumni-related information - such as personal details, degrees, achievements, job records, and donations. This leads to duplication, inconsistencies, and missing information.
- **Difficulty in Updating Records:** If an alumnus changes their phone number, job, or address, updating this information manually is slow and often forgotten. As a result, institutions frequently work with outdated or incomplete data.
- **Limited Search and Reporting Options:** Manual systems do not provide fast searching or filtering. For example, finding all alumni of the 2020 batch, or listing all donations made in a year, requires checking multiple files.
- **Security and Privacy Concerns:** Paper files or shared spreadsheets are not secure. Anyone may access or edit them without authorization. Sensitive information, such as contact details or donation records, can be easily misused.

5.2 Objectives

The main goals of the new system are:

- **Ease of Data Entry**: Allow administrators (and possibly alumni themselves) to easily add and update information using simple web forms. This eliminates handwritten or duplicate records.

- **Better Data Organization**: Store each type of information (degrees, projects, jobs, donations) in its own table, linked to the alumni. This avoids repeating data and makes it easy to see all details for one person.

- **Quick Search and Report**s: Enable fast search and filtering. For example, find all alumni in a certain year or list all donations in the last year without digging through papers.

- **Security and Privacy**: Provide login access, so only authorized users (admin) can change data. Alumni can possibly update their own profiles securely.

- **Improved Communicatio**n: By having up-to-date alumni records, the college can send announcements or newsletters more easily. (Although sending emails is beyond this project, storing correct contact info helps future communication.)

- **Scalability**: Use reliable tools so the system can grow (handle many alumni records) and be upgraded. React, Node, and MySQL can handle many users and lots of data efficiently.

By meeting these objectives, the system should save time and reduce mistakes compared to the old way. For example, instead of searching through files for all alumni who donated last year, an admin can run a quick database query. Also, eliminating redundant data entry improves data quality.

5.3 Proposed System

In the proposed Online Alumni Data Management System, all alumni-related tasks are performed on a computer via a website. Administrators (or alumni themselves) can log in and enter data using web pages instead of paper. The system will have modules for each function: a page to add or edit alumni personal information, and pages for each of the entities like Degrees, Achievements, Projects, Jobs, and Donations. When the admin fills in a form (for example, adding a new degree for an alumnus), React will send that information to the Node backend, which then stores it securely in the MySQL database.

All data will be organized: each alumnus is one record in the Alumni table. That alumni's degrees are stored in the Degree table (with a link back to the Alumni table), their jobs in the Job table, and so on. This means the system can automatically group information (for example, show all jobs for a selected alumnus). The system eliminates many manual steps. For example, retrieving an alumnus's full profile is done by clicking on their name, not by searching through paper files. Similarly, listing all donors is a quick database lookup.

Overall, the proposed system is more efficient, letting users get what they need with a click, and it fixes problems of the old system by keeping data consistent and centralized. As one project report on an alumni portal note, such systems usually have separate user roles (Admin and Alumni) for secure access, and they provide a user-friendly interface for registration and data updates. That is the idea behind our system: secure login and dynamic forms will ensure data is entered correctly, and the computer handles all the organization behind the scenes.

5.4 ENTITY - RELATIONSHIP DIAGRAM

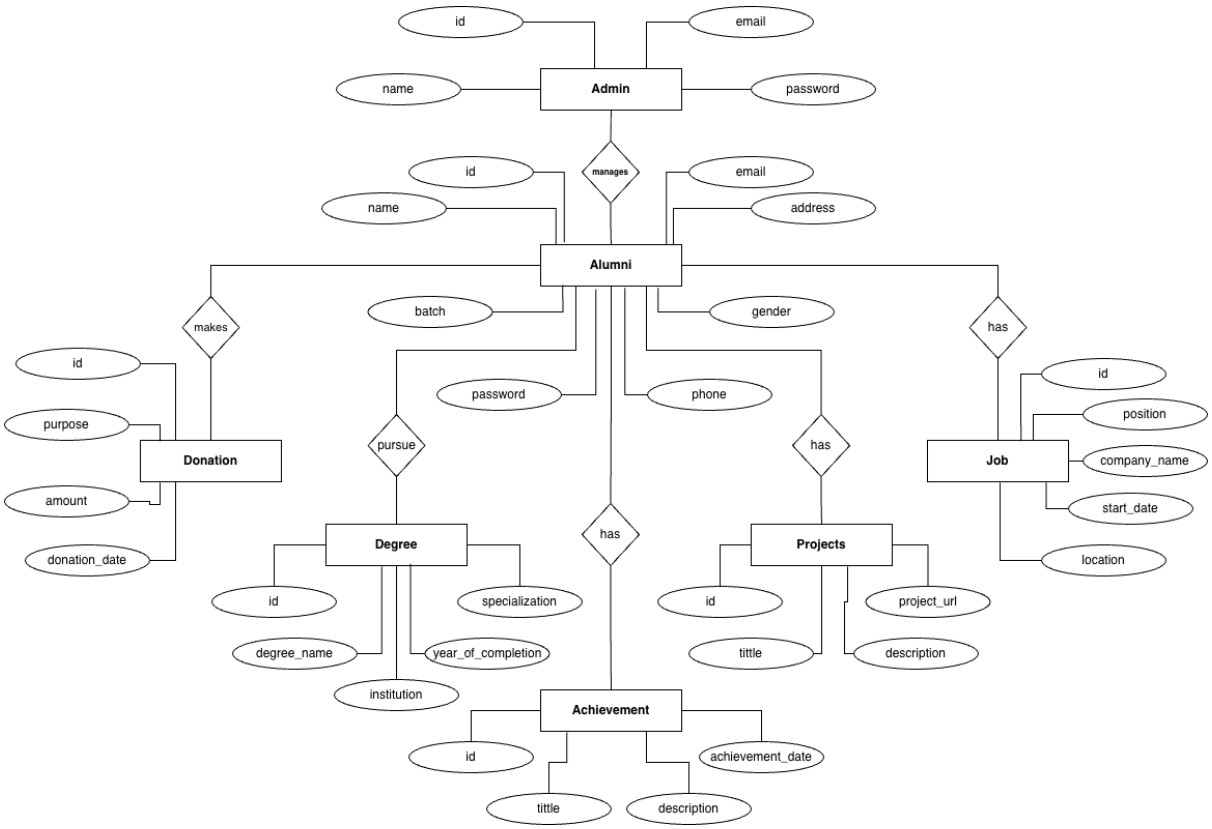


Fig 4.4 (a): Proposed Entity-Relationship Diagram

5.5 Relational Schema

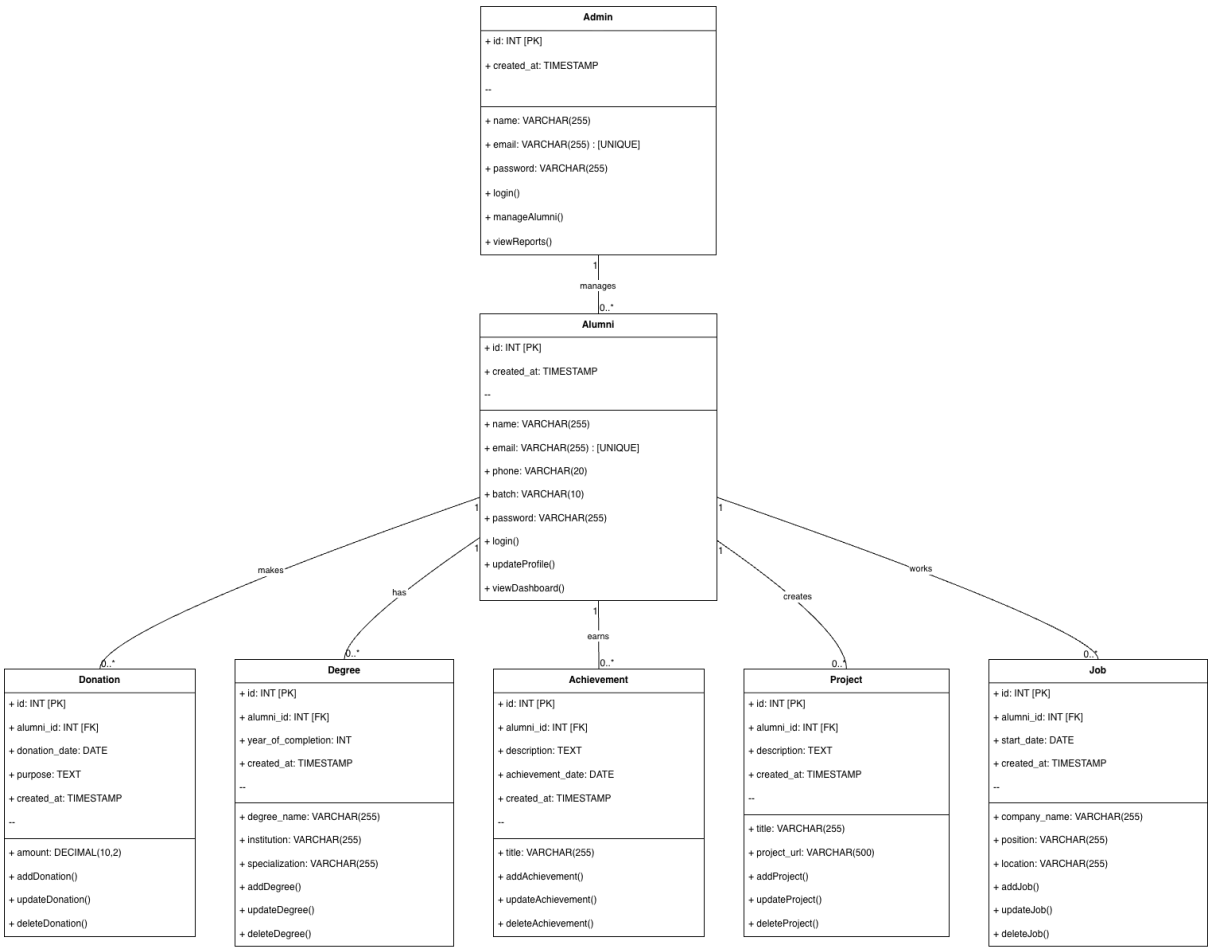


Figure 4.5 (a): Relational Database

5.5.1 Relational Schema

The relation schema for the given tables can be summarized as follows:

Admin

- id (PK)
- name
- email (unique)
- password
- created_at

Alumni

- id (PK)
- name
- email (unique)
- phone
- batch
- password
- created_at

Degree

- id (PK)
- alumni_id (FK → Alumni.id)
- degree_name
- institution
- year_of_completion
- specialization
- created_at

Achievement

- id (PK)
- alumni_id (FK → Alumni.id)
- title
- description
- achievement_date
- created_at

Project

- id (PK)

- alumni_id (FK → Alumni.id)
- title
- description
- project_url
- created_at

Job

- id (PK)
- alumni_id (FK → Alumni.id)
- company_name
- position
- start_date
- location
- created_at

Donation

- id (PK)
- alumni_id (FK → Alumni.id)
- amount
- donation_date
- purpose
- created_at

Relationships

a. **Alumni — Degree** : One-to-Many. (One Alumni can have many Degrees.)
b. **Alumni — Achievement** : One-to-Many. (One Alumni can have many Achievements.)
c. **Alumni — Project** : One-to-Many. (One Alumni can have many Projects.)
d. **Alumni — Job** : One-to-Many. (One Alumni can have many Jobs.)
e. **Alumni — Donation** : One-to-Many. (One Alumni can make many Donations.)
f. **Admin** : separate entity for system users with full rights. (Admins manage data; not directly FK-linked to alumni records in current schema.)

1. Admin

Column Name	Data Type	Constraints	Description
id	INT	PK, AUTO_INCREMENT, NOT NULL	Unique admin ID (primary key)
name	VARCHAR(255)	NOT NULL	Admin full name
email	VARCHAR(255)	NOT NULL, UNIQUE	Admin email (used for login)
password	VARCHAR(255)	NOT NULL	Hashed password
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Time when admin account was created

Table 5.5 (a): Admin Table

2. Alumni

Column Name	Data Type	Constraints	Description
id	INT	PK, AUTO_INCREMENT, NOT NULL	Unique alumni ID (primary key)
name	VARCHAR(255)	NOT NULL	Alumni full name
email	VARCHAR(255)	NOT NULL, UNIQUE	Alumni email (unique)
phone	VARCHAR(20)	NULL	Contact phone number
batch	VARCHAR(10)	NULL	Graduation batch/year (e.g., 2020)
password	VARCHAR(255)	NOT NULL	Hashed password for alumni login
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Time when record was created

Table 5.5 (b): Alumni Table

3. Donation

Column Name	Data Type	Constraints	Description
id	INT	PK, AUTO_INCREMENT, NOT NULL	Unique donation ID
alumni_id	INT	NOT NULL, FK → Alumni(id) ON DELETE CASCADE	Links donation to an alumni
amount	DECIMAL(10,2)	NOT NULL	Donation amount (currency)
donation_date	DATE	NULL	Date of donation
purpose	TEXT	NULL	Purpose / note about the donation
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	When the donation record was added

Table 5.5 (c): Donation Table

4. Degree

Column Name	Data Type	Constraints	Description
id	INT	PK, AUTO_INCREMENT, NOT NULL	Unique degree record ID
alumni_id	INT	NOT NULL, FK → Alumni(id) ON DELETE CASCADE	Which alumni earned this degree
degree_name	VARCHAR(255)	NOT NULL	Degree title (e.g., B.Tech)
institution	VARCHAR(255)	NOT NULL	Name of institution / college
year_of_completion	INT	NULL	Year degree was completed
specialization	VARCHAR(255)	NULL	Specialization or major
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	When the degree record was added

Table 5.5 (d): Degree Table

5. Achievement

Column Name	Data Type	Constraints	Description
id	INT	PK, AUTO_INCREMENT, NOT NULL	Unique achievement ID
alumni_id	INT	NOT NULL, FK → Alumni(id) ON DELETE CASCADE	Which alumni achieved this
title	VARCHAR(255)	NOT NULL	Short title of the achievement
description	TEXT	NULL	Detailed description
achievement_date	DATE	NULL	Date of the achievement
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	When the achievement was recorded

Table 5.5 (e): Achievement Table

6. Project

Column Name	Data Type	Constraints	Description
id	INT	PK, AUTO_INCREMENT, NOT NULL	Unique project ID
alumni_id	INT	NOT NULL, FK → Alumni(id) ON DELETE CASCADE	Which alumni did the project
title	VARCHAR(255)	NOT NULL	Project title/name
description	TEXT	NULL	Short description of the project
project_url	VARCHAR(500)	NULL	Link to project (GitHub, demo, etc.)
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	When the project was added

Table 5.5 (f): Project Table

7. Job

Column Name	Data Type	Constraints	Description
id	INT	PK, AUTO_INCREMENT, NOT NULL	Unique job record ID
alumni_id	INT	NOT NULL, FK → Alumni(id) ON DELETE CASCADE	Which alumni held this job
company_name	VARCHAR(255)	NOT NULL	Employer / company name
position	VARCHAR(255)	NOT NULL	Job title / position
start_date	DATE	NULL	Job start date
location	VARCHAR(255)	NULL	Job location (city, country)
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	When the job record was added

Table 5.5 (g): Job Table

5.6 Implementation

1. **Auth / Login Page**
   - Purpose: Let Admins or Alumni sign in using email + password.

2. **Register Page (Sign up)**
   - Purpose: Create new accounts (role can be alumni or admin).
   - Fields: role (alumni/admin), name, email, phone (alumni only), batch (alumni only), password.

3. **Dashboard (Overview)**
   - Purpose: Show summary stats (Total Alumni, Donations count, Achievements count, Projects count) and recent alumni list and quick overview text.
   - Elements: stats cards, recent alumni list, quick notes.

4. **Alumni List & Profile**
   - Purpose: View/search all alumni and their basic details. Admins can add, edit, delete alumni. Alumni can edit their own profile.
   - UI: Search box, table with Name / Email / Phone / Batch / Actions. Under each alumni row you show their degrees as a nested row.
   - Actions: Add Alumni (admin only), Edit (admin or owner), Delete (admin only).

5. **Donations Page**
   - Purpose: Show donation cards, add/edit/delete donation records.
   - Card fields: amount, donation_date, purpose, alumni info.

6. **Achievements Page**
   - Purpose: Show achievement cards, add/edit/delete achievement entries.

7. **Projects Page**
   - Purpose: Show project cards (title, description, URL), add/edit/delete projects.

8. **Jobs Page**
   - Purpose: Show job cards (company_name, position, start_date, location), add/edit/delete job entries.

9. **Degrees Page**

- Purpose: Show degree cards/records, add/edit/delete degrees.

5.7 Description of Modules

The system is divided into the following main modules or sections:

- **Admin Login Module:** A secure login page for administrators. Admins enter a username (or email) and password. The system checks these against the Admin table in MySQL. If valid, the admin gains access to the rest of the site.

- **Alumni Registration Module:** A page with a form to add a new alumnus. Fields include name, graduation year, department, email, contact number, etc. When the admin submits the form, the data is stored in the Alumni table. If any field is missing or invalid, the form shows an error message.

- **Alumni Management Module:** This includes pages to **view** and **edit** alumni records. The "Alumni List" page displays all alumni in a table, each with "Edit" and "Delete" buttons. On "Edit", a form pops up (or a new page opens) where the admin can change details. Submitting updates the Alumni table. Deleting removes that alumnus (after a confirmation).

- **Degree Module:** A page to enter or view degrees. When adding a degree, the admin first selects an alumnus (from a dropdown or search). Then enter degree details (type, branch, year). The record is saved in the Degree table, linked to that alumnus. There's also a "View Degrees" page showing all degrees for a selected alumnus.

- **Achievement Module:** Works similarly to Degrees. Each entry has fields like "Title of Achievement" and "Year". The admin can add achievements for an alumnus and see a list of them. All data is stored in the Achievement table.

- **Project Module:** Allows recording projects done by alumni. Fields might include project title, description, and year. Each project is linked to an alumni ID. The admin can add new projects or list existing ones for an alumnus.

- **Job Module:** Records job history of alumni. A form asks for company name, job title, start date, end date, etc. These are saved in the Job table. Admin can list or edit job entries per alumnus.

- **Donation Module:** Lets the admin record donations made by alumni. The form fields include the alumni ID (or selection), amount donated, and donation date. These go into

the Donation table. The admin can also view all donations, filter by alumni, or delete incorrect entries.

5.8 Execution Snapshots

5.8.1 Frontend

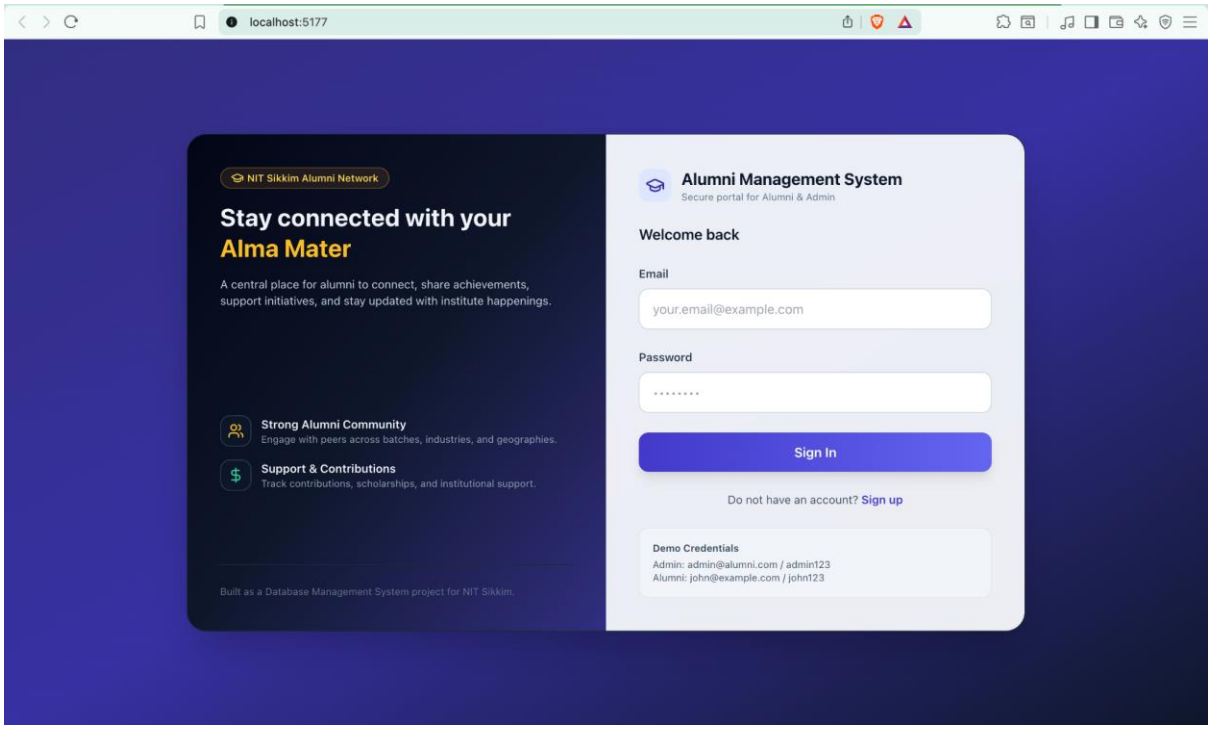


Figure 5.8.1 (a): Login Page

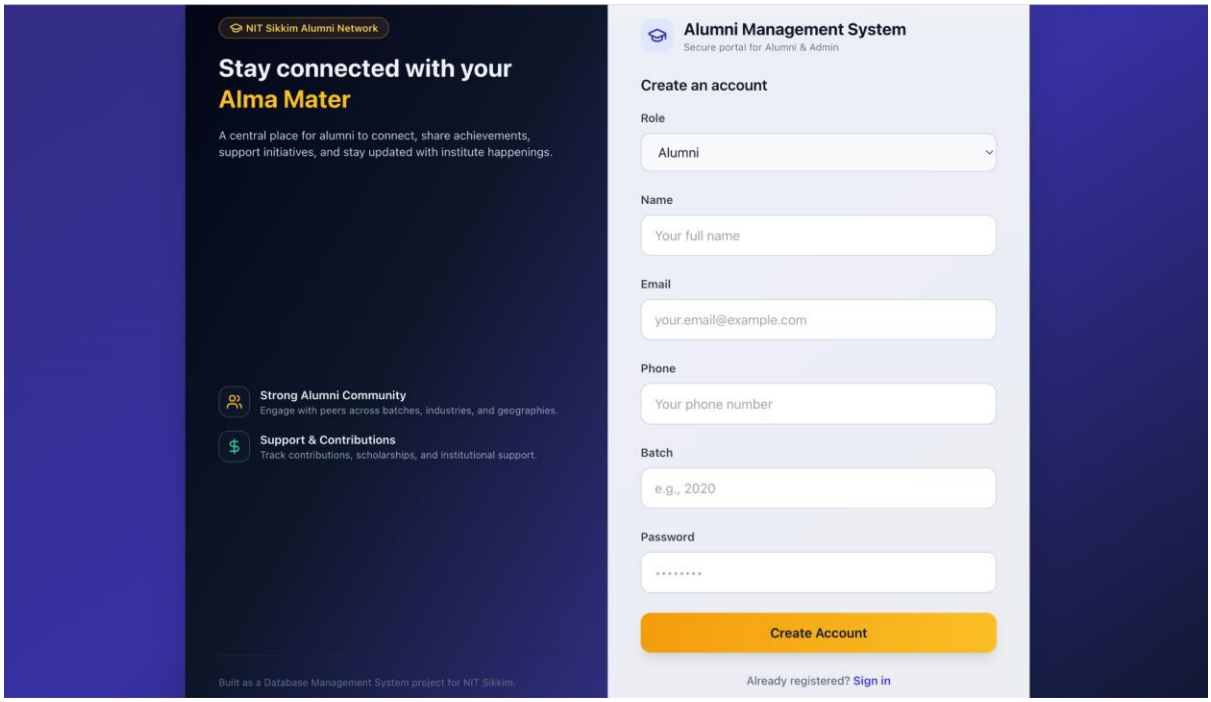


Figure 5.8.1 (b): Sign Up Page

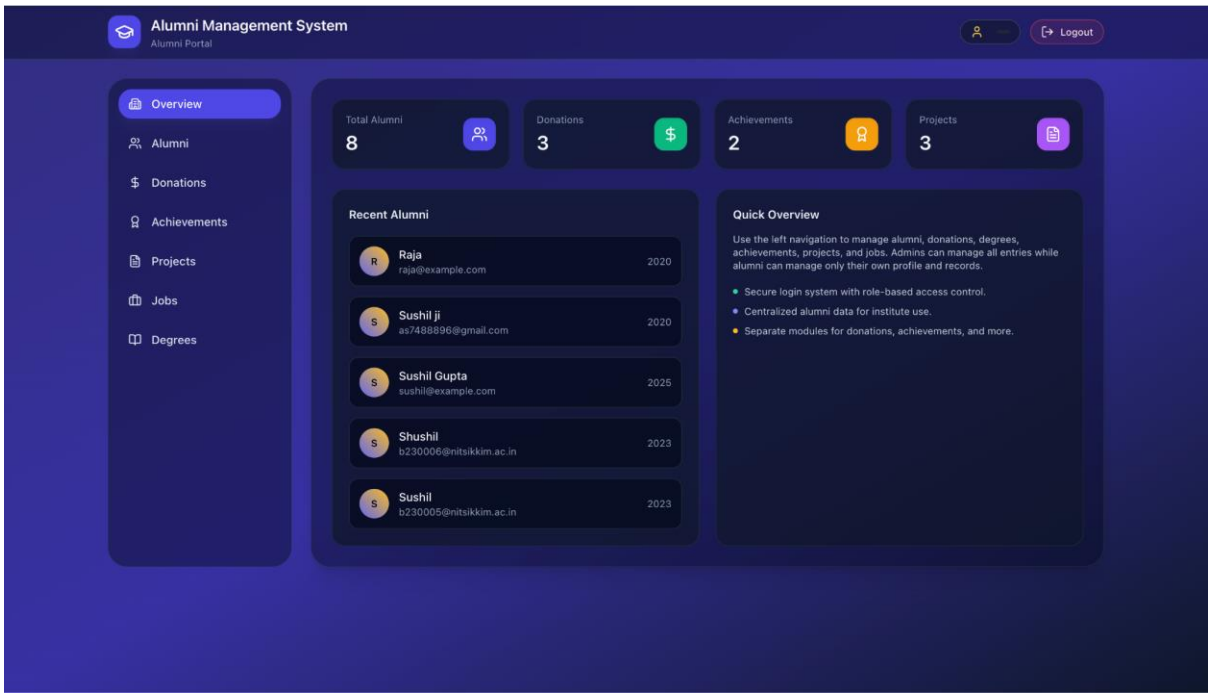


Figure 5.8.1 (c): Admin Dashboard

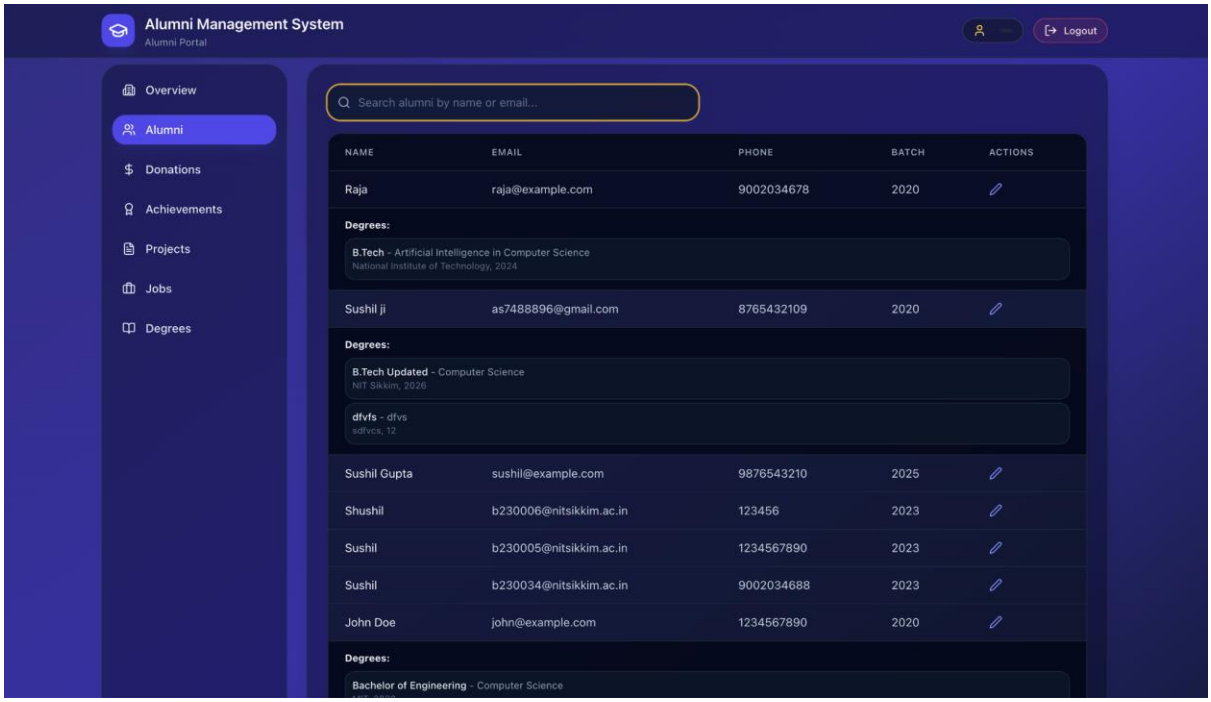
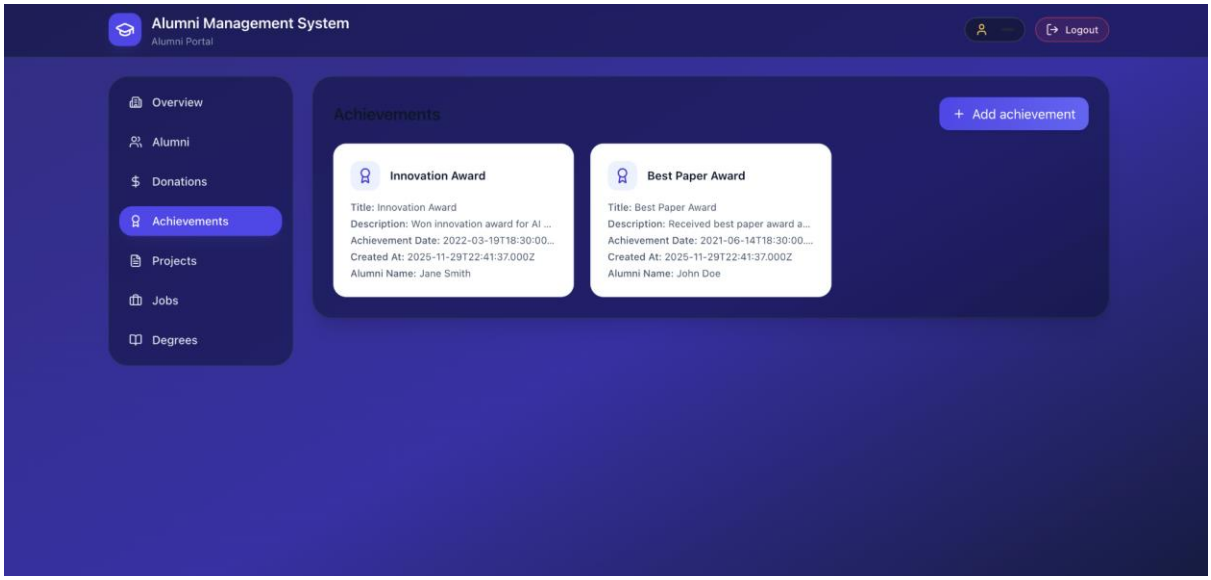


Figure 5.8.1 (d): Alumni List Page



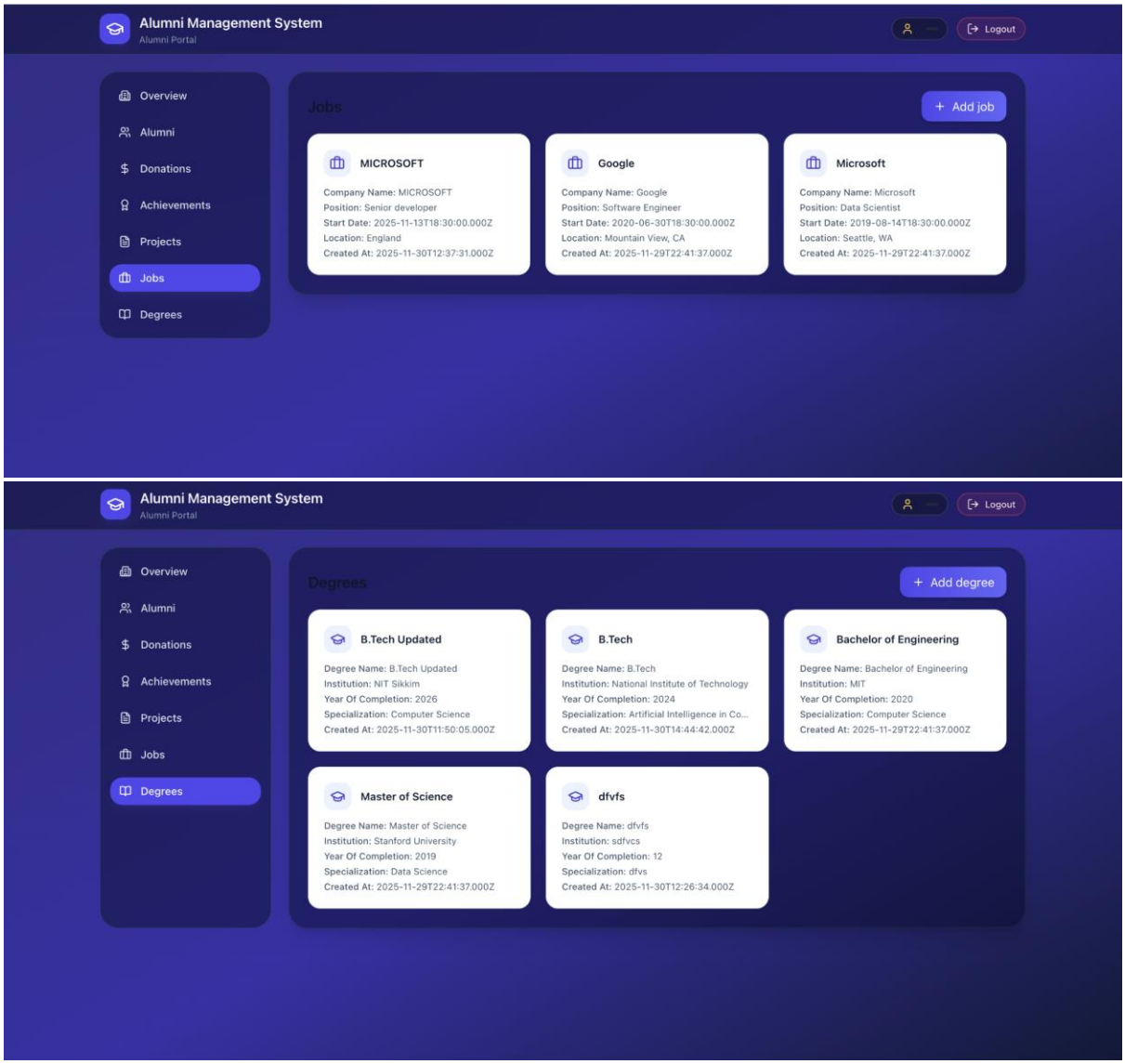


Figure 5.8.1 (e): Data Views

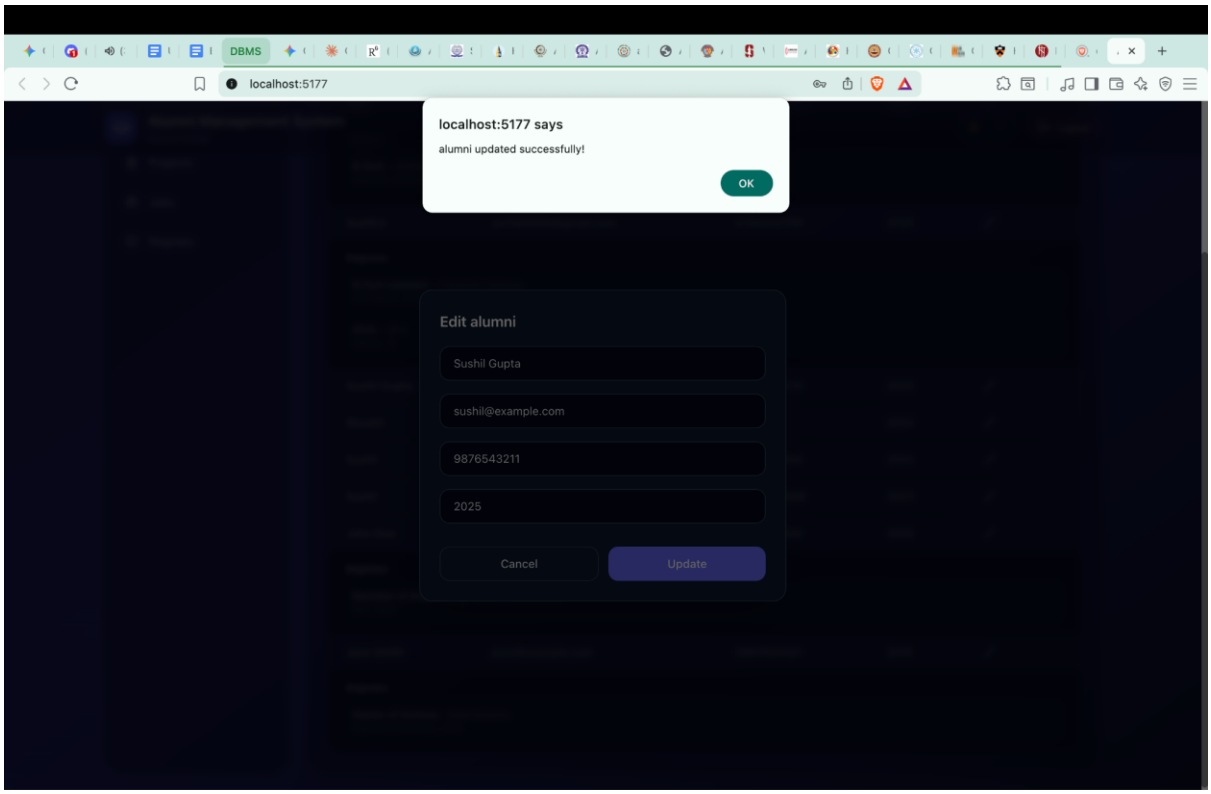


Figure 5.8.1 (f): Success message

5.8.2 Backend

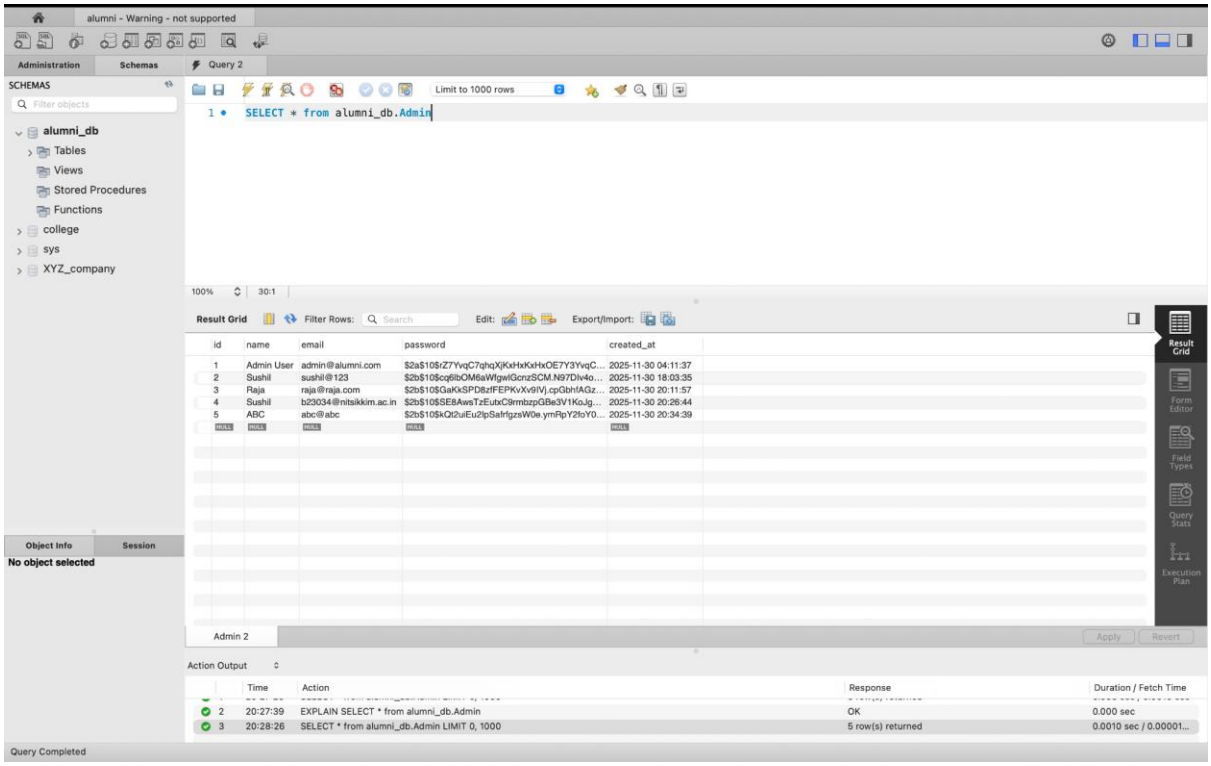


Figure 5.8.2 (a): Admin Database

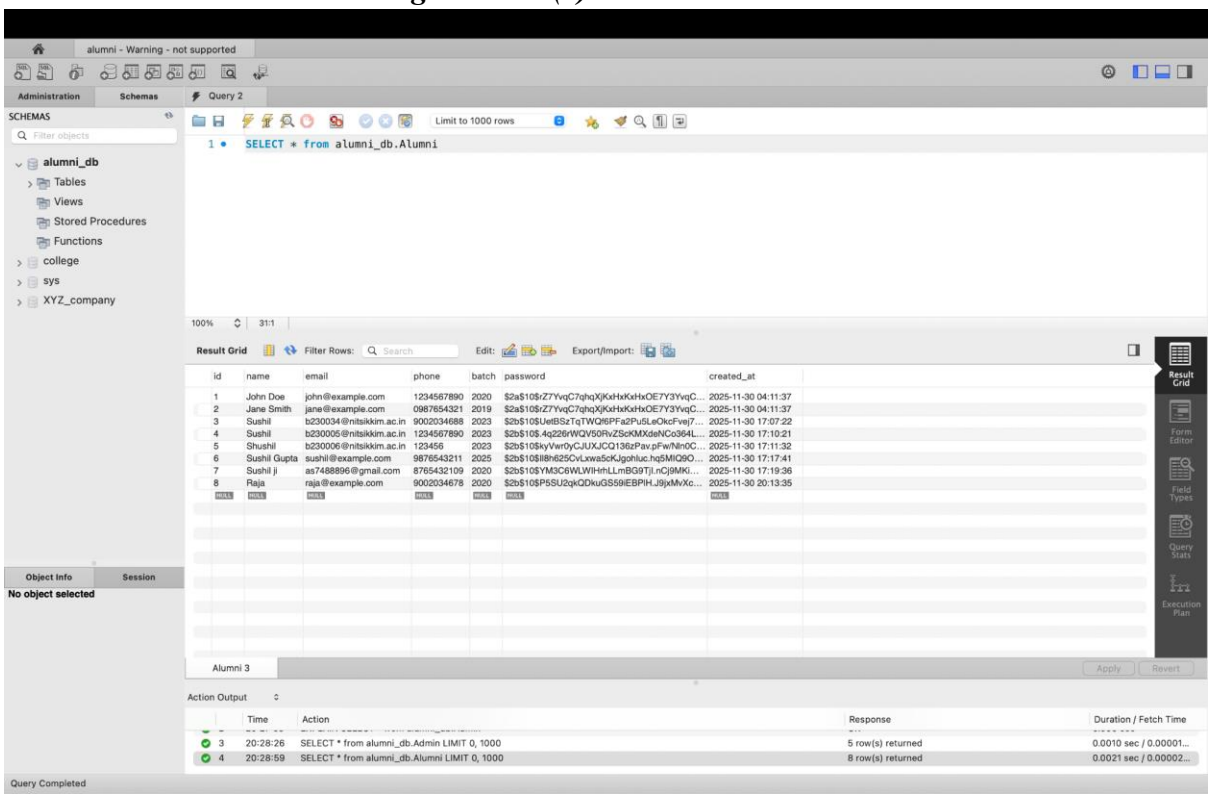


Figure 5.8.2 (b): Alumni Database

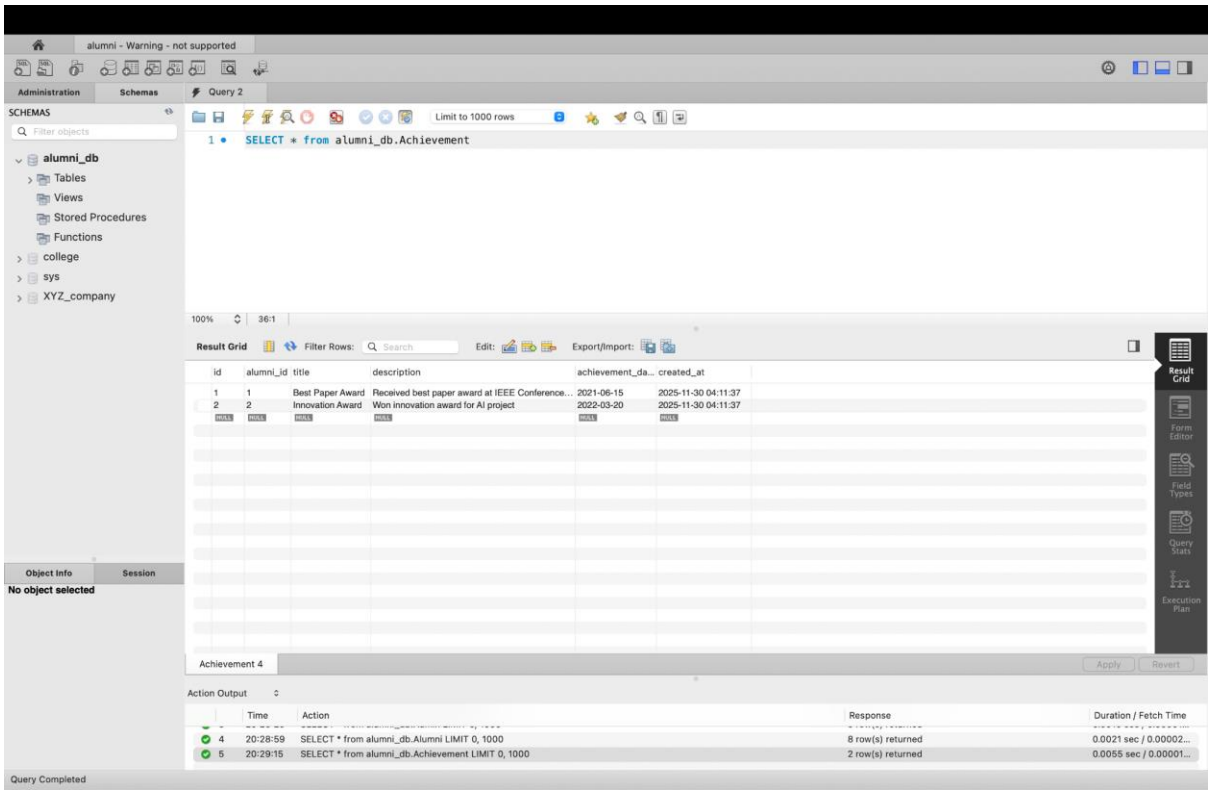


Figure 5.8.2 (c): Achievement Database

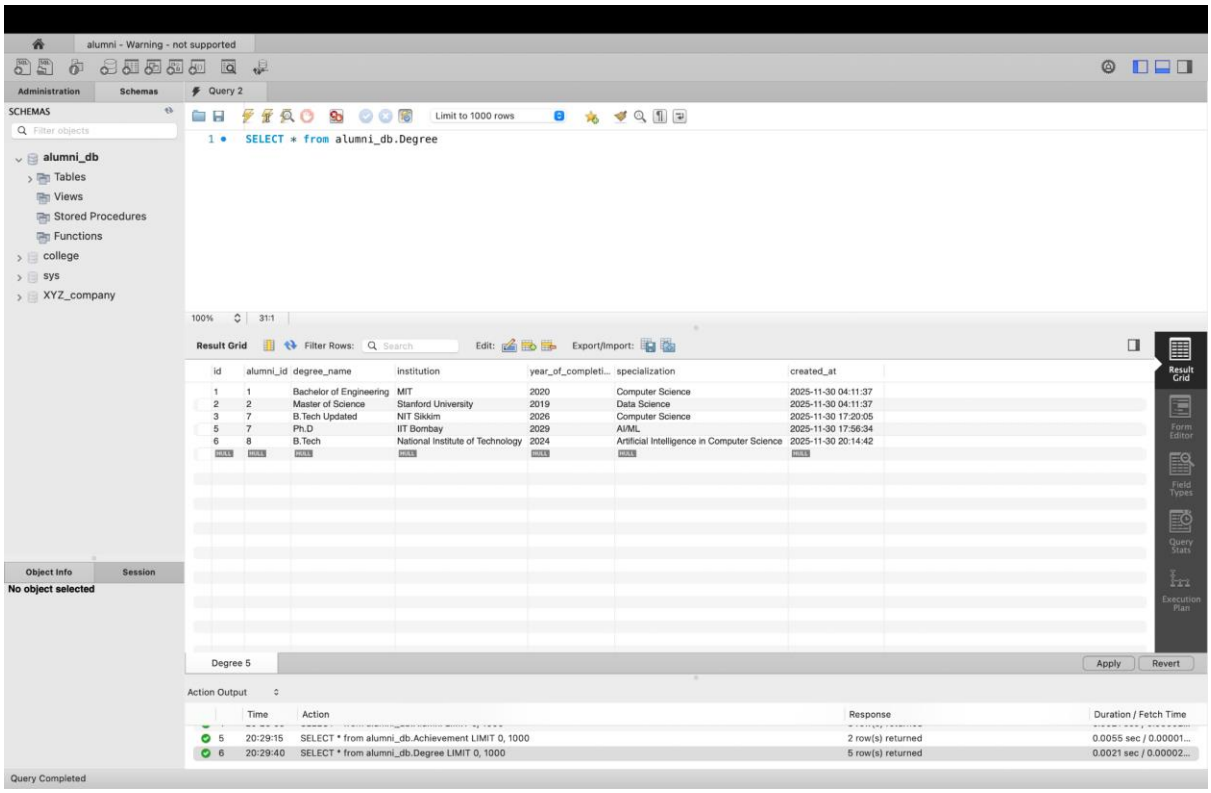


Figure 5.8.2 (d): Degree Database

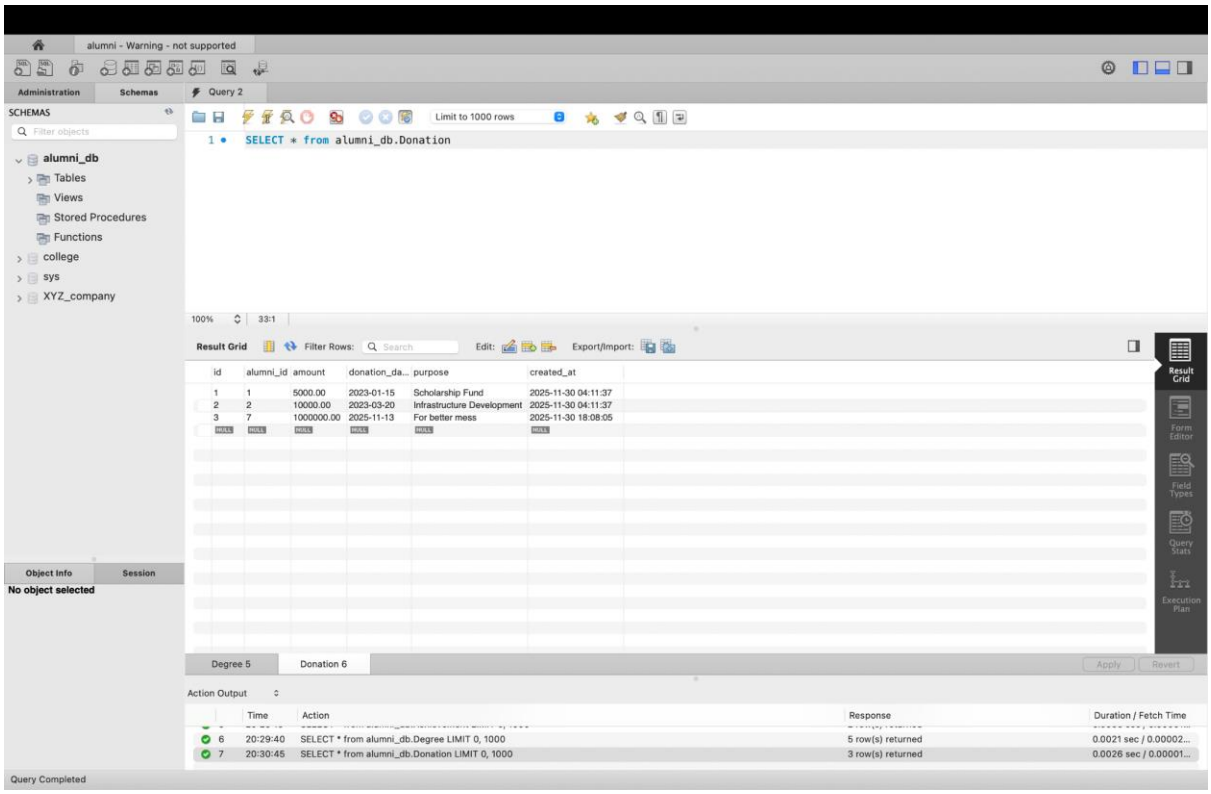


Figure 5.8.2 (e): Donation Database

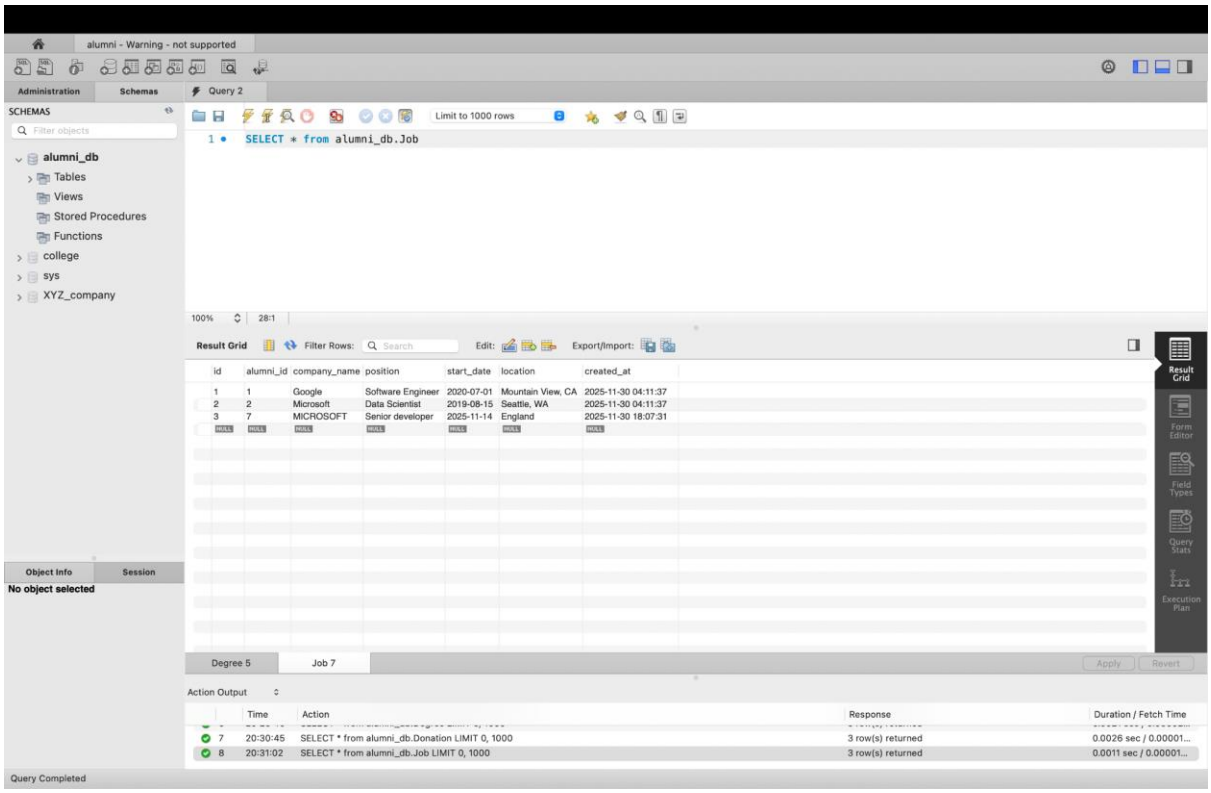


Figure 5.8.2 (f): Job Database

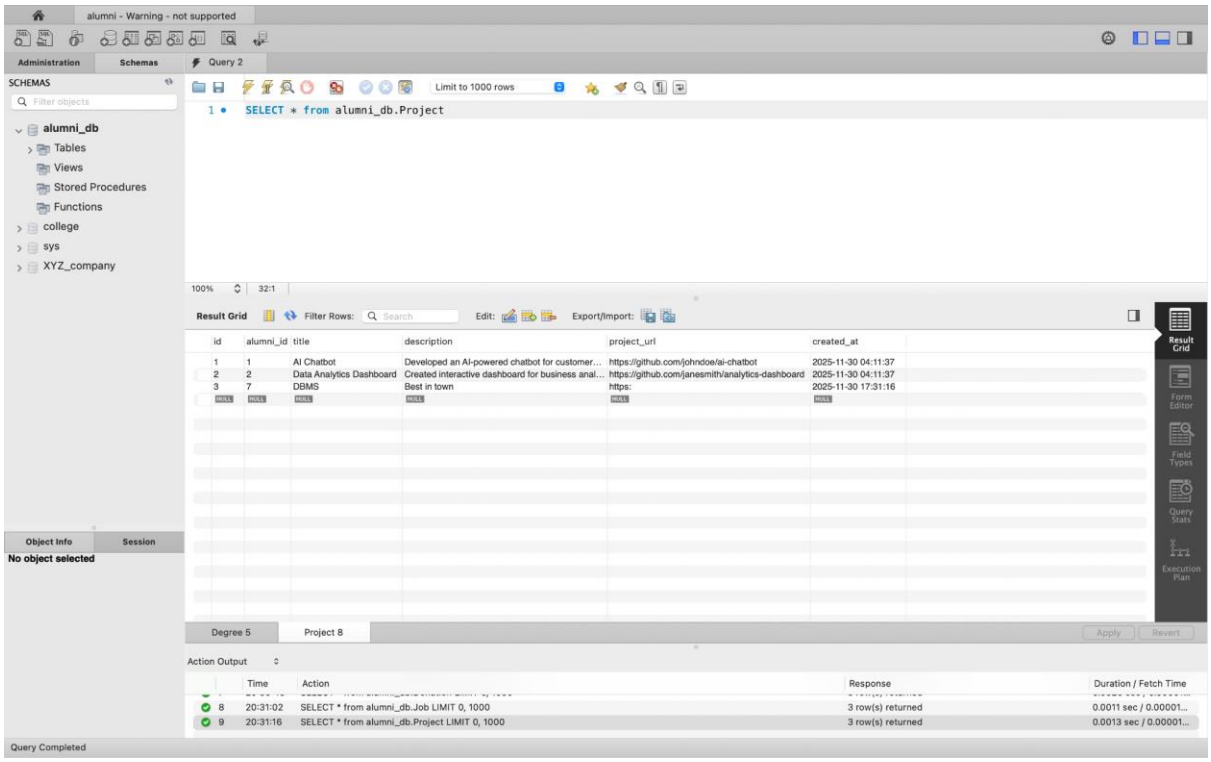


Figure 5.8.2 (f): Project Database

CHAPTER - 06

APPLICATIONS

An alumni management system offers these key real-life benefits:

- Store and manage alumni contact information (email, phone, batch) in one central place.
- Track alumni education (degrees) and professional history (jobs) for placement or networking.
- Record and report donations (amount, date, purpose) to support fundraising and accounts.
- Save and showcase alumni achievements and projects for publicity and institute documents.
- Provide role-based access: admins manage all data while alumni can view/update their own profile.
- Quick search and filter alumni by name, email, or batch for event invites or outreach.
- Support mentorship and networking by making alumni profiles and job/project details discoverable.
- Produce simple summary dashboards (counts of alumni, donations, achievements, projects) for meetings.
- Serve as a directory for students and staff to verify alumni credentials and achievements.
- Adaptable to non-profits or other organizations to manage former members, volunteers, or donors.

In short, any organization that benefits from tracking people's past involvement, qualifications, and contributions can use such a system.

CHAPTER - 07

CONCLUSION & FUTURE DIRECTIONS

The project provides a complete online solution for alumni management: a responsive React + Vite frontend and a Node.js/Express backend that talks to a MySQL database. The app supports user authentication, registration, role-aware UI (admin vs alumni), and full CRUD for all core entities (Alumni, Degree, Achievement, Project, Job, Donation). The frontend offers a clean interface: login/register, dashboard stats, searchable alumni list, modular cards for donations/achievements/projects/jobs/degrees, and a reusable modal for add/edit operations. The backend (as designed) enforces data storage and relationships (alumni ↔ degrees/achievements/projects/jobs/donations) using foreign keys and ON DELETE CASCADE, keeping data consistent.
Overall, the system replaces manual/fragmented records with a secure, centralized, and easy-to-use web application suitable for college use.

CHAPTER - 08

REFERENCES

Serial No.	Links
1	https://www.scribd.com/document/919535450/Alumni-management-pdf1#:~:text=Alumni%20management%20refers%20to
2	https://en.wikipedia.org/wiki/Node.js#:~:text=Node,loop%2C%20called%20the%20event%20loop
3	https://kinsta.com/blog/what-is-mysql/#:~:text=Recap%3A%20What%20is%20MySQL%3F
4	https://vite.dev/guide/#:~:text=Vite%20%28French%20word%20for%20,consists%20of%20two%20major%20parts
5	https://www.irejournals.com/formatedpaper/1702778.pdf#:~:text=2%20Professor%2C%20MCA%20Department%2C%20Modern,Controller
6	https://expressjs.com/#:~:text=Express%20is%20a%20minimal%20and,for%20web%20and%20mobile%20applications
7	https://ieeexplore.ieee.org/document/10142761